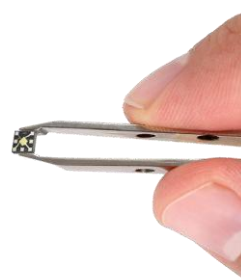


NANOZ

Demonstrator EK-II – EK-IV Communication Protocol Measurement Format



The UART serial port used for controlling and retrieving measurements has been set to a fixed rate of 921600 baud, while the USB port adapts to the speed chosen by the host application, except on the EK-IV which passes through the USB interface equipped with the FTDI module.

The protocol chosen is a simple ASCII command protocol.

Each command ends with either the <CR> (0x13) or <LF> (0x10) character.

Commands can be entered in uppercase or lowercase letters.

When binary data needs to be exchanged, such as when reading or writing parameters to non-volatile memory, a specific ASCII command is sent, indicating the length and checksum of the data block, which in turn will be passed immediately after the end-of-command character (<CR> or <LF>).

The checksum simply consists of an EXCLUSIVE OR of all the bytes that make up the transmitted data block (only the data).

The implemented commands are:

- **ver<CR>**
 Management: Host to Demonstrator
 Description: Returns the firmware version number as:

```
.....
      NANOZ EK gen IV SW:V1.12.153
      S/N: 0000-0007
      .....
```
- **whoami<CR>**
 Management: Host to Demonstrator
 Description: Returns the signature of the configuration block that allows identify the type of demonstrator:

```
Iam 5162<LF> for EK-II
Iam 5163<LF> for EK-III
Iam 5164<LF> for EK-IV
```
- **run <nn><CR>**
 Management: Host to Demonstrator
 Description: Request for the execution of cycle n° <nn>
 Parameters:
 <nn> The number of the cycle to be executed.
 Answer: Cycle #nn starting... <LF>

- Note: If no cycle has been defined with this identifier, or if a cycle is already running, an error message is returned.
- `pause<CR>`
 Management: Host to Demonstrator
 Description: Suspends the execution of the current cycle
 Answer: `Cycle nn completed!<LF>`
 Note: If no cycle is running, an error message is dismissed.
 - `calib!<CR>`
 For use on the EK-IV only with a kit of mounted 10K resistors. !
 - `calib ?<CR>`
 Management: EK-IV only: returns the values of the offsets measured during calibration
 Answer: `CAL-1: 0.123400 CAL-2: 0.567800`
 - `cleep<CR>`
 Management: Host to Demonstrator
 Description: Erase sequences and cycles in non-volatile memory
 Answer: `EEPROM cleared<LF>`
 - `rdeep <addr> <len> <CR>`
 Management: Host to Demonstrator
 Description: Reading a parameter block in non-volatile memory
 Parameters:
 <addr> The address of the first byte to be read, expressed in the number of bytes. This address is not necessarily on a page border
 <Len> The number of bytes to be transmitted.
 Answer: `block #eep! (see below)`
 - `#eep! <len> <cs><addr><CR>< data>`
 Management: Demonstrator to Host
 Description: A parameter block is transmitted from non-memory volatile
 Parameters:
 <Len> The number of bytes to be transmitted.
 <cs> checksum of the data block.
 <addr> The address of the first byte, expressed in the number of bytes. This address is not necessarily on a page border
 <data> A binary data block of length <len>.
 - `wreep <len> <cs><addr><CR>< data>`
 Direction: Host to Demonstrator
 Description: Write a parameter block to non-volatile memory
 Parameters:
 <Len> The number of bytes to write.
 <cs> checksum of the data block.
 <addr> The address of the first byte to be written, expressed in the number of bytes. This address is not necessarily on a page border

<data> A binary data block of length <len> to be written to the flash.

Answer: none if the operation is successful, error message otherwise.

Note: If less than <len> bytes were transmitted within one second of sending the <CR> character, or if a checksum error is detected, the write is not completed and an error message is sent.

- #env? <CR>

Management: Host to Demonstrator

Description: Request for Environment Data

Answer: Block #env! (see below)

- #envX! <len> <cs><time><bfr><CR>< data>

Direction: Demonstrator to Host

Description: Environment data in response to a request from the Host

Parameters:

X X=2 for EK-II kits and X=3 for EK-III and EK-IV kits

<Len> The number of bytes to be transmitted.

<cs> checksum of the data block.

<time> The number of milliseconds since startup.

<bfr> Number of bytes waiting in the transmission buffer

<data> A binary data block of length <Len> corresponding to a block of environmental measurements.

Note: The Environmental Data Block is refreshed internally every second, but is only transmitted on demand. The structure is as follows:

Variable	Type	No. bytes	Description
PPS	Unsigned integer	4	Number of seconds since the demonstrator was started
ADC_SamplesNB	Unsigned integer	2	Number of samples acquired in the second (must be equal to 1000)
ADC_Mask	Unsigned integer	2	Mask determining the channels that may have been acquired and transmitted. The four low-order bits each represent one of the channels. The bit is set to 1 if acquisition was possible. Must be equal to 0x0F
ADC_Samples	Unsigned integer	2 x 4 x 2	Number of samples acquired for each of the channels of each of the chips.
ADC_Voltage	Singles	2 x 4 x 4	Average voltage at the output of the instrumentation amplifier for each of the 4 channels of each of the chips.
ADC_Current	Singles	2 x 4 x 4	Average current calculated according to the gauge for each of the 4 channels of each chip

HTR_Voltage	Singles	2 x 2 x 4	Voltages applied to each of the heaters for each of the chips
ADC_MidValue	Single	4	Value measured in mV of the midpoint (virtual mass). Must be close to 2.25 V for the EK-II and 1.65V for the EK-III
MCU_Temperature	Single	4	Temperature internal of the microcontroller
Humidity_x100	Whole signed	2	Moisture in hundredths of a percent
tempH_x100	Whole signed	2	Temperature Outcome of the Sensor humidity in hundredths of a degree
Pressure_x10	Whole signed	2	Atmospheric pressure difference in tenth of a hectopascal compared to 1013hPa
tempP_x100	Whole signed	2	Temperature from the pressure sensor in hundredths of a degree
pending	Whole signed	2	Number of bytes pending in the Transmission buffer
Align	Whole signed	2	Block alignment bytes
Age	Unsigned Eniter	2x4	EK-IV specific: Duration of use of each of the Chips in seconds

Which can be defined in C language as follows:

```
packed struct env_typ {
    uint32_t    pps;
    uint16_t    ADC_SamplesNb;
    uint16_t    ADC_Mask;
    uint16_t    ADC_samples[SENSORS_NB][CHIPS_NB];
    float    ADC_Voltage[SENSORS_NB][CHIPS_NB]; float
    ADC_Current[SENSORS_NB][CHIPS_NB]; float
    HTR_Voltage[HEATERS_NB][CHIPS_NB]; float
    ADC_MidValue;
    float    MCU_Temperature;
    int16_t    humidity_x100; //0..100
    int16_t    tempH_x100; // -40..65
    int16_t    pressure_x10; //1013+/-
    int16_t    tempP_x100; // -40..65
    int16_t    pending;
    int16_t    align;
    uint32_t    Age[CHIPS_NB];
};
```

- #spl! <len> <cs><chip><time><bfr><CR>< data>
 Direction: Demonstrator to Host Description:
 Host-initiated measures Parameters:
 <Len> The number of bytes to be transmitted.
 <cs> checksum of the data block.
 <chip> Bullet Index 0 or 1

<time> The number of milliseconds since startup.
<bfr> Number of bytes waiting in the transmission buffer

<data> binary data frame of length <len> corresponding to a block of measures.

Note: Blocks of measurements are transmitted at the scheduled frequency when the sequence of a cycle is active. A block is sent for each of the two chips. The structure of the measurement block is as follows:

Variable	Type	No. bytes	Description
ppms	Unsigned integer	4	Number of milliseconds since the demonstrator was started
chipID	Unsigned integer	1	Number of the selected chip for this sequence (1 to 4)
sensorMask	Unsigned integer	1	Mask determining the sensors that may have been acquired and transmitted. The four low-order bits each represent one of the channels. The bit is set to 1 if acquisition was possible. Must be equal to 0x0F
Reserved	Unsigned integer	2	Alignment bytes
DAC_Voltage	Whole signed	4 x 2	Bias voltage in millivolts of each sensor
ADC_Current	Singles	4 x 4	Current Measured in milliamperes for each sensor
Heater_1_Voltage	Single	4	Voltage in millivolts applied to heating 1
Heater_1_Current	Single	4	Current in milliamperes flowing through the heater 1
Heater_2_Voltage	Single	4	Voltage in millivolts applied to heating 2
Heater_2_Current	Single	4	Current in milliamperes flowing through heating 2

Which can be defined in C language as follows:

```
typedef struct {
    float Voltage;
    float Current;
} heater_t;

__Packed struct sample_t {
    uint32_t ppms;
    uint8_t chipID;
    uint8_t sensorMask;
    uint16_t reserved;
    int16_t DAC_Voltage[SENSORS_NB];
    float ADC_Current[SENSORS_NB];
    heater_t heater[HEATERS_NB];
};
```